

Cara Kerja AI Auto Clipper

Penjelasan Alur Sistem — Versi Sederhana

Gambaran Besar

Fase 1 — Upload

Fase 2 — Klik “Process” → Job Dibuat

Fase 3 — Pipeline Otomatis (di background thread)

1. Extract
2. Transcribe
3. Analyze — bagian paling ramai
4. Score
5. Pilih Kandidat Terbaik

Fase 4 — Anda Pilih & Generate Clip (manual)

Bagaimana Data Mengalir Antar Tabel

Ringkasan Alur (Versi Super Singkat)

Gambaran Besar

Ini aplikasi **local web app** (bukan aplikasi yang jalan sendiri di background terus-menerus) — Anda buka browser, upload video, klik proses, lalu server mengerjakan semuanya bertahap di belakang layar sambil Anda pantau progress-nya.

Ada dua fase besar:

1. **Fase otomatis** (jalan sendiri setelah Anda klik “Process”): extract → transcribe → analyze → score → pilih kandidat terbaik.
 2. **Fase manual** (Anda yang pilih): lihat daftar kandidat → pilih mana yang mau dijadikan clip beneran → generate.
-

Fase 1 — Upload

Yang Anda lakukan: drag file video atau paste link YouTube di halaman upload.

Yang terjadi di belakang layar:

- Browser kirim file ke POST /upload (kalau file lokal) atau POST /upload/download (kalau dari URL).
 - VideoService (kalau upload) atau DownloadService (kalau dari URL) menyimpan file ke folder data/uploads/.
 - Satu baris baru masuk ke tabel videos di database — statusnya uploaded.
-

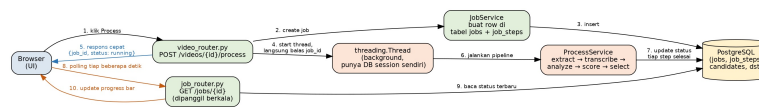
Fase 2 — Klik “Process” → Job

Dibuat

Yang Anda lakukan: klik tombol proses di video yang sudah diupload.

Yang terjadi:

- POST /upload/videos/{id}/process dipanggil.
- JobService bikin satu baris baru di tabel jobs (status pending), sekaligus bikin daftar step di tabel job_steps (extract, transcribe, analyze, score, generate, export — semua status pending).
- Server langsung balas ke browser {"job_id": ..., "status": "running"} — **tidak menunggu proses selesai**. Proses sebenarnya jalan di **background thread** terpisah, supaya browser tidak nge-freeze.
- Browser lalu polling (nanya berkala) ke GET /jobs/{job_id} untuk update progress bar.

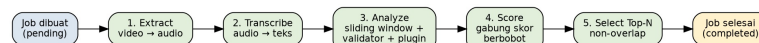


Alur request: browser, router, background thread, dan database

Poin pentingnya: langkah 4–5 di diagram di atas terjadi hampir bersamaan — server **tidak menunggu** pipeline selesai sebelum membalas browser. Itu sebabnya Anda langsung dapat job_id walau videonya belum selesai diproses, dan progress bar berjalan dari polling berkala, bukan dari satu request yang “digantung” sampai proses kelar.

Fase 3 — Pipeline Otomatis (di background thread)

Ini yang dikerjakan ProcessService, langkah demi langkah.



Lima langkah pipeline otomatis, dari job dibuat sampai selesai

1. Extract

- FFmpegService mengambil file video, memisahkan **audio-nya saja** jadi file terpisah (karena speech-to-text cuma butuh audio, bukan gambar).
- File audio disimpan ke cache (data/cache/), dicatat di tabel cache_entries supaya kalau diproses ulang tidak perlu extract lagi.

2. Transcribe

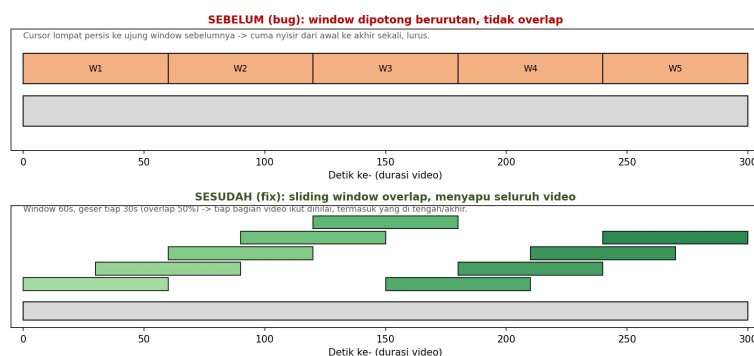
- TranscriptService kirim file audio ke **Whisper** (model speech-to-text yang jalan lokal di server Anda, bukan API luar).

- Hasilnya: teks lengkap ucapan di video plus potongan per-kalimat dengan timestamp masing-masing (disimpan di tabel transcripts dan transcript_segments).

3. Analyze — bagian paling ramai

AnalysisService melakukan:

- **Bikin window**: video “disapu” pakai jendela geser (sliding window) yang saling tumpang tindih, dari awal sampai akhir — bukan dipotong rata berurutan.
- Untuk **tiap window**, dua jenis analisis jalan:
 - **Text validator** (validators.py): cek isi teks window — ada hook menarik? ada kata sponsor/CTA yang bikin skor turun (penalty)? dan sebagainya.
 - **Plugin analyzer** (ai_modules/): analyzer yang sudah didaftarkan (LLM content, face emotion, dst) dipanggil satu per satu lewat registry. Kalau salah satu analyzer tidak aktif (misal API key kosong), dia di-skip, bukan bikin error.
- Tiap hasil analisis (apa pun jenisnya) disimpan sebagai satu baris di tabel analysis_results — generik, satu tabel untuk semua jenis analyzer.
- Tiap window juga langsung jadi satu baris di tabel candidates (skornya masih kosong dulu).



Sliding window: cara lama vs cara yang sudah diperbaiki

Diagram di atas kenapa penting: cara lama memotong video lurus dari detik 0 sampai habis (window saling menempel, tidak overlap) — jadi terkesan “cuma lanjutan menit”, bukan hasil pencarian momen terbaik. Setelah diperbaiki, window saling tumpang tindih dan menyapu seluruh durasi video, sehingga bagian tengah dan akhir video juga benar-benar ikut dinilai, tidak cuma kebagian karena posisinya pas berurutan.

4. Score

- ScoreEngine mengambil semua analysis_results per window, mengalikan tiap kategori dengan bobotnya masing-masing (LLM 30%, hook 10%, story 15%, dan seterusnya — bisa diatur di config), lalu menjumlahkannya jadi final_score per candidate.

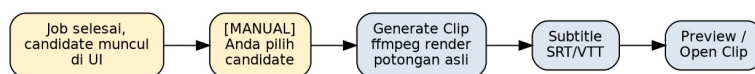
5. Pilih Kandidat Terbaik

- Dari semua window yang di-scan tadi (bisa puluhan), `select_top_n()` memilih yang skornya tertinggi **dan tidak saling tumpang tindih waktu**, sejumlah yang Anda minta (misal top 5).
- Sisanya (yang skor rendah/overlap) dihapus dari database.

Setelah step ini selesai, job berubah status jadi `completed`, dan daftar kandidat terbaik muncul di halaman **Candidate Clips**.

Fase 4 — Anda Pilih & Generate Clip (manual)

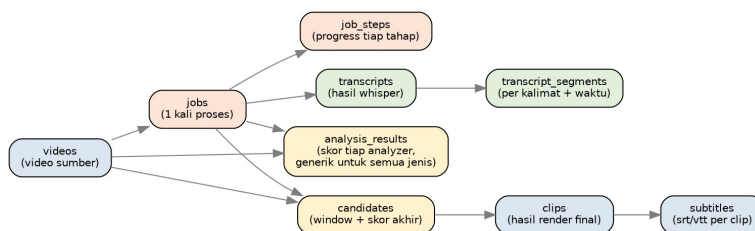
Pipeline otomatis **berhenti di sini**. Sisanya Anda yang kontrol:



Alur setelah job selesai: pilih candidate sampai preview clip

- Lihat tiap kandidat di halaman Candidates (start/end time, skor).
- Klik **“Generate Clip”** → POST `/clips` → ClipService panggil FFmpegService lagi, kali ini untuk benar-benar **merender/memotong video asli** sesuai window yang dipilih, simpan ke `data/outputs/`, dicatat di tabel `clips`.
- Kalau mau subtitle, SubtitleService men-generate file `.srt/.vtt` dari `transcript_segments` yang timestamp-nya masuk ke rentang clip itu.
- Klik **“Open Clip”** untuk preview hasil akhirnya.

Bagaimana Data Mengalir Antar Tabel



Relasi tabel database utama

Satu hal yang sengaja didesain generik: `analysis_results` adalah **satu tabel untuk semua jenis analyzer** (LLM, face emotion, voice emotion, hook, dst) — bukan satu tabel per analyzer. Jadi kalau nanti mau tambah analyzer baru (misal Music Analyzer), tidak perlu bikin tabel baru, cukup daftarkan jenis analyzer barunya.

Ringkasan Alur (Versi Super Singkat)

Upload/Download video
|
Klik Process -> job dibuat, jalan di background
|
Extract audio (ffmpeg)
|
Transcribe (whisper) -> teks + timestamp
|
Analyze: sapu seluruh video pakai sliding window,
 tiap window dinilai (text validator + AI analyzer plugin)
|
Score: gabungkan semua nilai jadi 1 angka per window
|
Pilih top-N terbaik yang tidak overlap -> jadi "candidates"
|
[MANUAL] Anda pilih candidate -> Generate Clip (ffmpeg render)
 -> Subtitle -> Preview